

TRABAJO DE FIN DE GRADO



ITV CyL (Nombre provisional)

CICLO FORMATIVO DE GRADO SUPERIOR EN
DESARROLLO DE APLICACIONES MULTIPLATAFORMA

DAVID ANDRÉS PÉREZ

JUAN FRANCISCO HERREROS GONZÁLEZ

ÍNDICE

1.	INTRODUCCIÓN	3
2.	FASE ANÁLISIS.....	4
2.1	Requisitos funcionales.....	4
2.2	Requisitos no funcionales.....	6
3.	FASE DISEÑO.....	8
3.1	Casos de Uso	8
3.1.1	Diagrama Casos de Uso	8
3.2	Análisis de persistencia	8
3.2.1	Diagrama Entidad-Relación	8
3.3	Diseño técnico	8
3.3.1	Arquitectura del sistema	8
3.3.2	Scripts desarrollados	9
3.4	Análisis diseño interfaz.....	16
3.4.1	Prototipo de Baja Fidelidad	16
3.4.2	Prototipo de Media Fidelidad.....	16
4.	PLAN DE PROYECTO	17
4.1	Cronograma.....	17
4.2	Reparto tareas.....	17
5.	PRUEBAS.....	18
5.1	Índice casos de prueba	18
5.2	Casos de prueba	18
6.	DOCUMENTACIÓN DE LA APLICACIÓN	19
6.1	Manual de instalación	19
6.2	Demo	19
7.	CONCLUSIONES	20
8.	USO DE LA IA	21
9.	BIBLIOGRAFÍA.....	22

1. INTRODUCCIÓN

El presente Trabajo de Fin de Ciclo recoge el desarrollo de ITV CyL, una aplicación de escritorio orientada a la gestión de inspecciones técnicas de vehículos en el ámbito de Castilla y León.

El proyecto surge como ejercicio práctico de integración de tecnologías — backend automatizado, base de datos relacional e interfaz de escritorio — aplicadas a un caso de uso real y cotidiano, centralizando tanto la gestión de los datos del parque vehicular como el registro de las inspecciones realizadas. Para ello, hemos desarrollado una solución que combina un backend automatizado en Python, desplegado en PythonAnywhere, con una aplicación de escritorio desarrollada en JavaFX que actúa como interfaz de usuario.

El backend se encarga de descargar mensualmente de forma automática los datos del parque de vehículos publicados por la Dirección General de Tráfico (DGT), filtrar los correspondientes a las nueve provincias de Castilla y León, asignar a cada vehículo una matrícula única generada mediante funciones de hash criptográfico, y volcar toda la información procesada en una base de datos MySQL alojada en PythonAnywhere. Este proceso, que gestiona cerca de dos millones de registros, se ejecuta de forma desatendida el día 5 y el día 20 de cada mes, garantizando que los datos del sistema estén siempre actualizados con la última información publicada por la DGT.

El frontend, desarrollado en JavaFX, permite a los usuarios del sistema — técnicos e inspectores de ITV y gerentes — acceder a la aplicación con sus credenciales y operar en función de su rol. Los técnicos pueden iniciar el proceso de revisión de un vehículo introduciendo su matrícula, lo que desencadena una consulta contra la base de datos que recupera los datos precargados del vehículo y, si procede, crea o vincula el registro del cliente correspondiente. A continuación, el técnico cumplimenta los datos propios de la inspección — estado de frenos, neumáticos, luces, electrónica, niveles de gases, resultado y fecha de la próxima revisión, entre otros — que quedan registrados en la base de datos asociados a dicha matrícula. Los gerentes, además de las funcionalidades de los técnicos, tienen acceso al área de administración de usuarios, donde pueden dar de alta y de baja a los miembros del equipo.

A lo largo de este documento describimos en detalle las fases de análisis y diseño del sistema, la arquitectura técnica desarrollada, las pruebas realizadas y las conclusiones obtenidas tras el proceso de desarrollo.

2. FASE ANÁLISIS

2.1 Requisitos funcionales

ID	Nombre	Descripción	Prioridad
RF1	Servidor en la nube	Disponer de una cuenta activa en PythonAnywhere que actúe como servidor remoto para ejecutar los scripts del sistema.	Alta
RF2	Base de datos en la nube	Crear y configurar una base de datos MySQL en PythonAnywhere accesible desde la aplicación de escritorio.	Alta
RF3	Descarga automática de datos DGT	Automatizar la descarga mensual de los archivos ZIP publicados por la DGT en su página oficial, detectando siempre la fecha más reciente disponible.	Alta
RF4	Filtrado por provincias de CyL	Extraer del ZIP descargado únicamente los TXT correspondientes a las nueve provincias de Castilla y León.	Alta
RF5	Procesamiento de archivos	Procesar los TXT extraídos, limpiar los valores incorrectos o vacíos, y generar un CSV por provincia con los campos seleccionados.	Alta
RF6	Generación de matrículas únicas	Asignar a cada vehículo una matrícula única generada mediante funciones de hash criptográfico (MD5), distinguiendo entre formato moderno (NNNNLLL) y antiguo (PPNNNNLL) según el año de primera matriculación.	Alta
RF7	Control de colisiones de matrícula	Garantizar que no existen matrículas duplicadas en el sistema. En caso de colisión, añadir un sufijo numérico a la matrícula y marcarla como Descartada (D) en el campo ESTADO_ACTUAL.	Alta
RF8	Inserción en base de datos	Insertar los datos procesados en la tabla vehiculos_cyl de MySQL, gestionando altas, duplicados y bajas de forma incremental mes a mes.	Alta
RF9	Detección automática de bajas	Detectar los vehículos que ya no aparecen en los datos de la DGT respecto al mes anterior y actualizar su ESTADO_ACTUAL a Baja (B) en la base de datos.	Alta
RF10	Script coordinador	Disponer de un script principal (coordinador.py) que orqueste la ejecución secuencial de todos los scripts del sistema, deteniéndose ante cualquier error.	Alta
RF11	Automatización mensual	Configurar una tarea programada en PythonAnywhere (Scheduled Tasks) que ejecute el coordinador diariamente, actuando únicamente los días 5 y 20 de cada mes.	Alta

RF12	Ejecución manual forzada	Permitir la ejecución manual del coordinador en cualquier momento mediante el parámetro --forzar, ignorando el control de días.	Media
RF13	Tabla vehiculos_cyl	La tabla de vehículos debe contener los campos: MATRICULA, PROVINCIA, MARCA, MODELO, FECHA_MATR, PROCEDENCIA, NUEVO_USADO, TIPO_DGT, TIPO_DISTINTIVO, EMISIONES_CO2, PLAZAS, FECHA_ACTUALIZACION y ESTADO_ACTUAL.	Alta
RF14	Tabla usuarios	Crear la tabla de usuarios con los campos necesarios para gestionar el acceso al sistema: identificador, nombre, apellidos, correo, contraseña y rol (técnico o gerente).	Alta
RF15	Tabla clientes	Crear la tabla de clientes con los campos: DNI, nombre, apellidos, teléfono, correo, dirección y matrícula asociada.	Alta
RF16	Tabla inspecciones	Crear la tabla de inspecciones asociada a una matrícula y a un cliente, con los campos: kilómetros actuales, ITV en vigor, niveles de gases, estado de frenos, estado de electrónica, luces defectuosas, neumáticos defectuosos, frenos defectuosos, intermitentes defectuosos, resultado de la inspección, fecha de la siguiente revisión y observaciones.	Alta
RF17	Autenticación de usuarios	La aplicación debe mostrar una pantalla de login como primera vista al arrancar. El acceso al sistema requiere credenciales válidas en todos los casos.	Alta
RF18	Roles de usuario	El sistema debe distinguir entre dos roles: técnico y gerente, con funcionalidades diferenciadas para cada uno.	Alta
RF19	Área personal	Todos los usuarios autenticados pueden consultar sus datos personales (nombre, cargo y correo electrónico) desde su área personal.	Alta
RF20	Gestión de usuarios	El gerente puede dar de alta y de baja usuarios del sistema desde su área personal. El técnico no tiene acceso a esta funcionalidad.	Alta
RF21	Revisión de vehículo	El sistema debe permitir iniciar el proceso de inspección introduciendo la matrícula del vehículo, que recupera automáticamente los datos precargados desde la base de datos.	Alta
RF22	Gestión de clientes	Al iniciar una inspección, el sistema debe comprobar si la matrícula tiene un cliente asociado. Si lo tiene, carga sus datos; si no, permite crear un cliente nuevo.	Alta
RF23	Registro de inspección	El sistema debe permitir cumplimentar y guardar los datos de la inspección, asociándolos a la matrícula y al cliente correspondientes.	Alta

RF24	Aplicación de escritorio	Desarrollar la interfaz de usuario mediante JavaFX, siguiendo los principios de usabilidad y claridad para un entorno profesional.	Alta
RF25	Manual de usuario	El proyecto debe incluir un manual de uso de la aplicación de escritorio dirigido a técnicos y gerentes.	Baja

2.2 Requisitos no funcionales

ID	Nombre	Descripción	Prioridad
RNF1	Rendimiento	El sistema debe ser capaz de procesar y gestionar cerca de dos millones de registros mensuales en un tiempo razonable, sin comprometer la estabilidad del servidor.	Alta
RNF2	Disponibilidad	El servidor de PythonAnywhere debe estar operativo en los días de ejecución automática (días 5 y 20 de cada mes) para garantizar la actualización de los datos.	Alta
RNF3	Seguridad en el acceso	El acceso a la aplicación debe requerir autenticación obligatoria. Las contraseñas deben almacenarse de forma segura en la base de datos, nunca en texto plano.	Alta
RNF4	Integridad de los datos	El sistema debe garantizar que no existen registros duplicados en la base de datos. Todas las inserciones y actualizaciones se realizan dentro de transacciones que se deshacen ante cualquier error.	Alta
RNF5	Unicidad de matrículas	Cada vehículo del sistema debe tener una matrícula única e irrepetible, generada de forma determinista a partir de sus datos técnicos.	Alta
RNF6	Trazabilidad	El sistema debe informar por pantalla del estado de cada paso del proceso (descarga, extracción, procesado e inserción), facilitando la detección de errores.	Media
RNF7	Mantenibilidad	El código debe estar estructurado en scripts independientes con responsabilidades bien definidas, comentado y documentado para facilitar su mantenimiento y evolución.	Alta
RNF8	Escalabilidad	El sistema debe estar diseñado para poder incorporar nuevas provincias o ampliar el conjunto de datos procesados sin necesidad de reescribir el código.	Media

RNF9	Compatibilidad	La aplicación de escritorio debe funcionar en los sistemas operativos más habituales en entornos empresariales (Windows, macOS y Linux).	Alta
RNF10	Usabilidad	La interfaz de usuario debe ser clara e intuitiva, permitiendo que un técnico sin conocimientos técnicos avanzados pueda operar el sistema sin formación específica.	Alta
RNF11	Almacenamiento	La base de datos MySQL debe ser capaz de almacenar el volumen de datos generado (estimado en torno a 500-600 MB) dentro de los límites del plan contratado en PythonAnywhere (5 GB).	Alta
RNF12	Codificación de caracteres	Todos los ficheros generados (CSVs) deben usar codificación UTF-8 con BOM para garantizar la correcta visualización de caracteres especiales como tildes y eñes en cualquier entorno.	Media
RNF13	Consistencia mensual	El sistema debe producir los mismos resultados ante los mismos datos de entrada, garantizando que un mismo vehículo recibe siempre la misma matrícula en cada ejecución mensual.	Alta
RNF14	Gestión de errores	Ante cualquier fallo en el proceso, el sistema debe informar del error con el mayor detalle posible y detener la ejecución de forma controlada, sin dejar datos a medias en la base de datos.	Alta

3. FASE DISEÑO

3.1 Casos de Uso

Añadir a partir de la reunión del jueves 16

ID	Descripción	Prioridad
----	-------------	-----------

3.1.1 Diagrama Casos de Uso

3.2 Análisis de persistencia

Añadir a partir de la reunión del jueves 16

3.2.1 Diagrama Entidad-Relación

3.3 Diseño técnico

3.3.1 Arquitectura del sistema

El sistema se estructura en tres capas bien diferenciadas que trabajan de forma coordinada: el backend automatizado, la base de datos y la aplicación de escritorio.

El backend, alojado en PythonAnywhere, es el encargado de todo el trabajo con los datos de la DGT. Cinco scripts de Python se ejecutan en cadena cada mes: el primero descarga el fichero de datos de la web oficial de la DGT, el segundo extrae únicamente los datos de las provincias de Castilla y León, el tercero los limpia y genera las matrículas únicas, el cuarto sube todo a la base de datos, y el quinto — el coordinador — orquesta a los otros cuatro y se encarga de que el proceso se lance automáticamente los días 5 y 20 de cada mes.

La base de datos MySQL, también en PythonAnywhere, almacena la información de los vehículos, los usuarios del sistema, los clientes y las inspecciones realizadas. Actúa como punto de unión entre el backend y el frontend.

La aplicación de escritorio, desarrollada en JavaFX, es la herramienta que usan los trabajadores en su día a día. Se conecta a la base de datos para consultar los datos de los vehículos, registrar inspecciones y gestionar los usuarios del sistema.

En cuanto a PythonAnywhere — base sobre la que se desarrolla nuestro trabajo — , durante el desarrollo del proyecto trabajamos inicialmente con la versión gratuita de la plataforma, lo que nos generó varios problemas que condicionaron algunas decisiones técnicas. El más significativo fue que todas las conexiones HTTP y HTTPS de la cuenta gratuita pasan por un proxy compartido,

lo que provocaba que sitios como la página de la DGT bloquearan nuestras peticiones devolviendo errores de tipo ProxyError o 403 Forbidden. Esto hacía imposible automatizar la descarga de los datos directamente desde el servidor, obligándonos a descargar el ZIP en local y subirlo manualmente a PythonAnywhere en cada ejecución.

Además, la versión gratuita impone restricciones significativas en cuanto al almacenamiento de la base de datos, limitado a 512 MB, lo que resultaba insuficiente para el volumen de datos que manejamos, cercano a los 500-600 MB estimados. Las tareas programadas también presentaban limitaciones, ya que solo permite configurar tareas con una frecuencia mínima diaria y no ofrece la flexibilidad necesaria para un proceso mensual controlado.

La migración al plan de pago, concretamente el plan Hacker de 10 dólares al mes, resolvió todos estos problemas de raíz. El acceso directo a internet sin restricciones de proxy nos permitió automatizar completamente la descarga de datos desde la página de la DGT. El límite de almacenamiento de la base de datos se amplió a 5 GB, muy por encima de nuestras necesidades actuales y con margen suficiente para crecer. Y la funcionalidad de tareas programadas nos permitió configurar el coordinador para que se ejecute automáticamente cada día y actúe únicamente los días 5 y 20 de cada mes, completando así el ciclo de automatización que buscábamos desde el principio.

Meter gráfico como los del trabajo

3.3.2 Scripts desarrollados

Hay que darle una vuelta. Creo que la explicación de los primeros ha quedado muy técnica poniendo los nombres literales de las funciones, variables, etc., mientras que los últimos son más explicativos que técnicos.

Descarga.py

El primer script del pipeline es `descarga.py`, cuya única responsabilidad es detectar y descargar el archivo ZIP más reciente publicado por la Dirección General de Tráfico con los datos del parque de vehículos mensual.

Para llevar a cabo su función, el script hace uso de cuatro librerías. La primera es `os`, que permite trabajar con rutas y carpetas del sistema de ficheros. La segunda es `re`, que proporciona soporte para expresiones regulares y nos permite buscar patrones de fecha dentro del HTML de la DGT. La tercera es `requests`, con la que realizamos tanto la descarga de la página web como la del archivo ZIP. Y la cuarta es `BeautifulSoup`, que interpreta el HTML descargado y nos permite extraer el texto visible de la página de forma sencilla.

Al inicio del script se definen tres constantes que centralizan la configuración. `RUTA_TEMP` indica la carpeta donde se guardarán los archivos ZIP descargados. `URL_DGT` contiene la dirección oficial de la página de la DGT donde se publican los listados mensuales. Y `PATRON_ZIP` define la estructura de la URL del archivo ZIP, con un marcador de fecha que se sustituirá automáticamente por la fecha detectada en cada ejecución.

La función `asegurar_carpeta()` se encarga de garantizar que la carpeta de destino existe antes de intentar guardar ningún archivo en ella. Para ello comprueba si la ruta definida en `RUTA_TEMP` existe y, en caso contrario, la crea automáticamente. Es una comprobación sencilla pero necesaria para evitar errores al guardar el ZIP en una ruta inexistente.

La función `obtener_ultima_fecha()` es la más relevante del script. Su objetivo es detectar automáticamente la fecha del archivo más reciente publicado por la DGT, sin necesidad de que nadie la introduzca manualmente. Para ello descarga el HTML de la página oficial y verifica que la petición ha sido correcta, de forma que si hay cualquier error en la descarga se lanza una excepción en lugar de continuar con datos incorrectos. A continuación, BeautifulSoup extrae todo el texto visible del HTML descartando etiquetas y código, y sobre ese texto aplicamos una expresión regular para encontrar todas las cadenas que coincidan con el formato YYYYMM, por ejemplo 202602 para febrero de 2026. Si no se encuentra ninguna fecha, el script lanza una excepción controlada que detiene el proceso e informa del problema. Cuando hay varias fechas disponibles, obtenemos la más reciente usando simplemente la función `max()` de Python: con el formato YYYYMM, la comparación alfabética coincide exactamente con la comparación cronológica, por lo que `max()` devuelve siempre la fecha más reciente sin necesidad de ningún cálculo adicional.

Con esa fecha, la función `descargar_zip()` construye la URL exacta del archivo ZIP, genera el nombre del archivo y construye la ruta completa donde se guardará. La descarga se realiza con requests y, una vez verificada, el archivo se guarda en disco en modo binario, es decir, byte a byte sin ninguna transformación de codificación, ya que se trata de un archivo comprimido y no de texto plano. La función devuelve la ruta completa del ZIP descargado para que el siguiente script del pipeline pueda localizarlo.

Por último, la función `main()` centraliza el flujo del script ejecutando las tres funciones en el orden correcto. El bloque `if __name__ == "__main__"` garantiza que `main()` solo se ejecute cuando el script se lanza directamente. Si otro script lo importa, como es el caso del coordinador, las funciones quedan disponibles pero `main()` no se ejecuta de forma automática.

ProcesarTXT.py

El segundo script del pipeline es `procesarTXT.py`, cuya responsabilidad es abrir el archivo ZIP descargado por `descarga.py`, extraer únicamente los ficheros TXT correspondientes a las provincias de Castilla y León y eliminar el ZIP una vez terminado el proceso para liberar espacio en disco.

Para trabajar con archivos comprimidos, el script utiliza la librería `zipfile`, que permite abrir y manipular ficheros ZIP sin necesidad de herramientas externas. Además importa la función `obtener_ultima_fecha()` de `descarga.py`, lo que le permite detectar automáticamente qué ZIP debe procesar en cada ejecución sin que sea necesario indicarlo manualmente.

Al inicio del script se definen las constantes necesarias. `RUTA_ZIPS` indica la carpeta donde se encuentran los archivos ZIP descargados, y `RUTA_TXT` la carpeta donde se guardarán los ficheros extraídos. También se define la lista `PROVINCIAS_CYL` con los nombres de las nueve provincias de Castilla y León en el formato que utiliza la DGT en sus ficheros. Un detalle importante aquí es el tratamiento de las tildes: los archivos dentro del ZIP pueden aparecer nombrados con tilde o sin ella dependiendo del mes, por lo que en la versión definitiva del script aplicamos una normalización de caracteres usando la librería `unicodedata` antes de comparar, lo que nos permite comparar siempre sin tildes y cubrir cualquier variante que pueda publicar la DGT sin necesidad de duplicar entradas en la lista.

La función `descomprimir_zip()` es el núcleo del script. Abre el ZIP con `zipfile.ZipFile` y recorre todos los ficheros que contiene mediante `infolist()`. Para cada fichero, normaliza su nombre eliminando tildes y lo convierte a mayúsculas, de forma que la comparación con los nombres de

las provincias sea siempre consistente independientemente de cómo los haya nombrado la DGT ese mes. Si el nombre del fichero contiene alguna de las provincias de nuestra lista, se extrae en la carpeta de destino.

Para evitar extraer el mismo fichero más de una vez, utilizamos un conjunto llamado `extraídos` que va acumulando los nombres de los ficheros ya procesados. Antes de extraer cualquier fichero comprobamos que no esté ya en ese set, y si no lo está lo extraemos y lo añadimos. Esta comprobación es sencilla pero evita duplicados en situaciones donde el ZIP pudiera contener el mismo fichero en distintas rutas internas.

Una decisión técnica importante en este script es que no extraemos todos los ficheros del ZIP, sino únicamente los que nos interesan. El ZIP de la DGT contiene datos de todas las provincias de España, por lo que extraer solo las nueve de Castilla y León reduce significativamente el uso de almacenamiento y memoria, algo especialmente relevante en un entorno con recursos limitados como PythonAnywhere.

Una vez finalizada la extracción, el script comprueba si el archivo ZIP sigue existiendo en disco y, en caso afirmativo, lo elimina con `os.remove()`. De esta forma conservamos únicamente los TXT que necesitamos para el siguiente paso del pipeline, liberando el espacio que ocupaba el ZIP original, que puede llegar a ser considerable.

La función `main()` centraliza el flujo del script. Llama a `obtener_ultima_fecha()` para detectar la fecha del ZIP más reciente, construye el nombre del fichero y su ruta completa, y llama a `descomprimir_zip()` para realizar la extracción. Al igual que en el resto de scripts, el bloque `if __name__ == "__main__"` garantiza que `main()` solo se ejecute cuando el script se lanza directamente, permitiendo que sus funciones puedan importarse desde el coordinador sin ejecutarse automáticamente.

ProcesarCSV.py

El tercer script del pipeline es, sin duda, el más complejo del proyecto. Su función es leer los ficheros TXT extraídos por `procesarTXT.py`, procesar sus datos y generar nueve archivos CSV, uno por cada provincia de Castilla y León, listos para ser insertados en la base de datos. Pero antes de explicar cómo lo hace, conviene entender qué es lo que queremos conseguir y por qué.

El reto central de este script es asignar a cada vehículo una matrícula única, que lo identifique de forma inequívoca en el sistema, que sea la misma cada vez que se ejecute el proceso mensual, y que no dependa de ningún dato personal del propietario. Los ficheros de la DGT contienen únicamente características técnicas de los vehículos, sin ningún identificador personal, por lo que necesitábamos un mecanismo que generara un identificador estable a partir de esos datos técnicos.

La solución que adoptamos se basa en funciones de hash criptográfico: si cogemos los datos técnicos de un vehículo, los unimos en una cadena de texto y aplicamos una función matemática que siempre produce el mismo resultado para la misma entrada, obtenemos un identificador único y reproducible. Ese identificador es lo que llamamos hash, y el algoritmo que usamos es MD5, que convierte cualquier cadena de texto en una secuencia de 32 caracteres hexadecimales.

Sin embargo, surge un problema: dos vehículos pueden tener exactamente las mismas características técnicas. Por ejemplo, dos coches del mismo modelo, marca, motor y año de fabricación producirían el mismo hash, y por tanto no podríamos distinguirlos. Para resolverlo introducimos el concepto de instancia. Llevamos un contador por cada hash: el primer vehículo

con unas características determinadas es la instancia 0, el segundo es la instancia 1, el tercero la instancia 2, y así sucesivamente. Combinando el hash con el número de instancia obtenemos un segundo hash que sí es único para cada vehículo individual, ya que nunca dos registros pueden tener el mismo hash y el mismo número de instancia al mismo tiempo.

Con ese segundo hash en mano, generamos la matrícula. Los ficheros de la DGT contienen la fecha de matriculación de cada coche, por lo que nosotros distinguimos entre vehículos matriculados antes y después del año 2000, aprovechando esa distinción para replicar los dos formatos de matrícula que han existido en España. Los vehículos matriculados desde el año 2000 reciben una matrícula con el formato actual de cuatro dígitos y tres letras, mientras que los anteriores al año 2000 reciben el formato antiguo, en el que las dos primeras letras identifican la provincia. El alfabeto que usamos tiene 21 consonantes, excluyendo tanto la Ñ como las vocales, lo que nos permite generar combinaciones más próximas al aspecto de una matrícula real española y evitar combinaciones que podrían resultar inapropiadas.

Aun con todo este mecanismo, existe una probabilidad matemática pequeña pero real de que dos vehículos distintos generen la misma combinación de dígitos y letras. Esto ocurre porque estamos comprimiendo un hash de 128 bits en un espacio de combinaciones posibles limitado, y con un gran volumen de registros las colisiones son inevitables según la paradoja del cumpleaños. Para gestionarlo, llevamos un registro de todas las matrículas ya asignadas: si una matrícula ya existe, a la nueva se le añade un sufijo numérico, y esa matrícula queda marcada en el sistema como Descartada. El campo ESTADO_ACTUAL, que añadimos a cada registro, refleja este estado con los valores A para Activa, D para Descartada y B para Baja, aunque este último queda reservado para la fase de inserción en base de datos.

Una mención aparte merece la provincia de Ávila. A diferencia del resto de provincias de Castilla y León, Ávila no aparece de forma consistente en los archivos publicados por la DGT, ya que la institución trata de forma irregular aquellas provincias cuyo nombre contiene tildes, y en función del mes puede aparecer o no en el ZIP. Por este motivo, el conjunto de datos que manejamos corresponde a las ocho provincias restantes, quedando Ávila fuera del sistema hasta que la DGT resuelva esta inconsistencia en la publicación de sus datos. Sin embargo, para aquellos meses en los cuales la DGT inserte correctamente dicha provincia, se integrará automáticamente y en el mismo sentido que las otras ocho.

Podríamos poner también los aspectos matemáticos, es decir, el total de registros, y las colisiones que habrá.

Con todo esto claro, pasamos a explicar cómo está construido el script.

Para leer los ficheros TXT, el script utiliza la librería pandas, que permite cargar ficheros delimitados por el carácter de barra vertical y con codificación latin-1, que es la que usan los ficheros de la DGT. Pandas facilita enormemente el trabajo con tablas de datos y la generación posterior de los CSVs.

Antes de calcular ningún hash, cada valor de cada celda pasa por la función limpiar_valor(). Los datos de la DGT contienen con frecuencia valores vacíos, nulos, caracteres de control o símbolos sin sentido que, si no se normalizan, harían que dos vehículos idénticos generaran hashes distintos. Para evitarlo, aplicamos tres filtros en cascada: el primero convierte a cero cualquier valor que sea vacío, nulo o equivalente a nulo; el segundo descarta valores que contengan caracteres no imprimibles como el símbolo M-! que aparece en algunos registros como dato desconocido; y el tercero descarta valores que no contengan ninguna letra ni dígito, como símbolos solos. Con estos tres filtros garantizamos que los datos de entrada al hash son siempre

coherentes y comparables entre registros. El patrón de detección de valores vacíos se compila una sola vez al inicio del script para no repetir ese trabajo en cada una de las millones de llamadas a la función.

Para construir el hash no usamos todos los campos del fichero, sino únicamente los que identifican las características técnicas del vehículo de forma inmutable: provincia, marca, modelo, tipo, variante, fecha de primera matriculación, subtipo y tipo DGT, tara, peso máximo, cilindrada, potencia, propulsión, categoría eléctrica, tipo distintivo, emisiones, carrocería, distancia entre ejes, plazas y sus variantes. Excluimos deliberadamente campos como la fecha de matriculación actual, el tipo de titular o el número de titulares, porque no describen al vehículo en sí sino su situación administrativa, que puede cambiar sin que el vehículo cambie.

Una vez construido el hash de características, consultamos el mapa de instancias para saber cuántas veces hemos visto ese hash antes, obtenemos el número de instancia actual, incrementamos el contador para la próxima vez, y calculamos el segundo hash combinando el primero con el número de instancia. La función que convierte ese segundo hash en matrícula interpreta los 16 bytes del hash como un número entero enorme y extrae de él cada componente de forma independiente, sin consumir el número, para aprovechar al máximo toda la información que contiene el hash y minimizar así las colisiones.

Una vez generadas todas las matrículas, construimos el dataframe final con los campos que irán al CSV. Aplicamos tres conversiones de legibilidad: el código numérico de la provincia se convierte al nombre completo usando un diccionario, el código de procedencia se convierte a su descripción textual, y los valores N y U de nuevo o usado se convierten a Nuevo y Usado. Finalmente añadimos la fecha y hora de ejecución como campo de auditoría y el estado actual de cada matrícula.

Los CSVs se guardan con codificación UTF-8 con BOM, que es la variante de UTF-8 que necesita Microsoft Excel para interpretar correctamente los caracteres especiales como tildes y eñes. Sin esa marca, Excel abriría los ficheros con una codificación incorrecta y los caracteres aparecerían distorsionados.

Un aspecto que merece mención especial es el orden de procesado. Los ficheros se procesan siempre en orden alfabético, y el mapa de instancias y el registro de matrículas usadas son variables globales compartidas entre todos los ficheros. Esto garantiza que, ante los mismos datos de entrada, el proceso produce siempre los mismos resultados, lo que es fundamental para que el sistema sea predecible y mantenible mes a mes.

Por último, cabe destacar una decisión técnica importante relacionada con el rendimiento. Durante el desarrollo intentamos optimizar el recorrido de las filas usando la función `apply()` de pandas, que internamente es más eficiente que el bucle tradicional fila a fila. Sin embargo, descubrimos que `apply()` no garantiza el orden estricto de ejecución cuando hay efectos secundarios en variables globales como el mapa de instancias o el registro de matrículas, lo que provocaba que algunas matrículas aparecieran duplicadas sin el sufijo correspondiente. Por este motivo revertimos al bucle fila a fila con `iterrows()`, que aunque es algo más lento garantiza que el procesado es siempre secuencial y predecible.

InsertarBBDD.py

El cuarto script del pipeline es `insertarBBDD.py`, cuya responsabilidad es leer los nueve CSVs generados por `procesarCSV.py` e insertar su contenido en la base de datos MySQL. Este script

está diseñado para ejecutarse de forma segura mes a mes, detectando qué es nuevo, qué ya existe y qué ha desaparecido, y actuando en consecuencia en cada caso.

Para conectarse a MySQL utilizamos la librería `mysql-connector-python`, que es la librería oficial de MySQL y la más compatible con PythonAnywhere. Los datos de conexión, host, usuario, contraseña y nombre de la base de datos, están centralizados en un diccionario al inicio del script, lo que facilita modificarlos si cambian las credenciales sin necesidad de tocar el resto del código. Si la conexión falla por cualquier motivo, el script informa del error y se detiene de inmediato.

Antes de procesar ningún CSV, el script carga en memoria todas las matrículas que ya existen en la tabla `vehiculos_cyl` mediante una consulta `SELECT`. Las almacenamos en un conjunto de Python porque las búsquedas en un conjunto son prácticamente instantáneas independientemente del número de elementos que contenga, lo que es fundamental cuando manejamos un volumen de registros del orden de los millones. Este conjunto representa una fotografía del estado actual de la base de datos y es la referencia que usamos durante todo el proceso para saber qué existe y qué no.

Un aspecto que requirió atención especial fue el formato de las fechas. Los CSVs generados por `procesarCSV.py` almacenan las fechas en formato `DD/MM/YYYY`, que es el formato habitual en España, pero MySQL espera los campos de tipo `DATE` en formato `YYYY-MM-DD` y los de tipo `DATETIME` con la hora incluida. Para resolver esto implementamos una función de conversión que acepta ambos formatos, con hora y sin ella, y los transforma al formato correcto antes de cada inserción. Si el valor recibido es inválido o está vacío, la función devuelve un valor nulo para que MySQL lo almacene como tal, evitando errores de formato que abortarían el proceso.

Para cada fila de cada CSV, el script toma una decisión basada en la matrícula. Si la matrícula ya está en el conjunto de matrículas existentes en base de datos, el registro se salta sin hacer nada, ya que ese vehículo ya fue procesado en una ejecución anterior. Si la matrícula no está en el conjunto, el registro se inserta completo en la tabla con todos sus campos. Usamos la instrucción `INSERT IGNORE` en lugar del `INSERT` estándar como medida de seguridad adicional: si por algún motivo una matrícula duplicada llegara a la fase de inserción, MySQL la descartaría silenciosamente en lugar de abortar el proceso con un error.

Al mismo tiempo que procesamos cada fila, vamos acumulando todas las matrículas de los CSVs en un segundo conjunto global. Este conjunto nos sirve al final del proceso para detectar las bajas. Una vez procesados los nueve CSVs, comparamos ese conjunto con el de matrículas que había en la base de datos al inicio. Las matrículas que estaban en la base de datos pero no aparecen en ninguno de los CSVs del mes actual son vehículos que ya no están en los datos de la DGT, lo que interpretamos como que han causado baja. Para marcarlos actualizamos su campo `ESTADO_ACTUAL` al valor `B` con una única instrucción `UPDATE` que afecta a todas las bajas de una sola vez, lo que es mucho más eficiente que lanzar una consulta individual por cada vehículo dado de baja.

Todas estas operaciones, inserciones y actualizaciones de baja, se ejecutan dentro de una transacción. Si el proceso llega a su fin sin errores, se confirman todos los cambios con un `commit`. Si en cualquier punto se produce un fallo, se deshacen todos los cambios realizados en esa ejecución mediante un `rollback`, dejando la base de datos exactamente en el estado en que estaba antes de empezar. De esta forma garantizamos que la base de datos nunca queda a medias: o se completa el proceso íntegro o no se guarda nada. La conexión con la base de datos

se cierra siempre al final, independientemente de si el proceso terminó con éxito o con error, gracias al bloque `finally` que envuelve todo el flujo principal.

Gracias a este diseño, el script puede ejecutarse mes a mes de forma segura y predecible. La primera vez inserta todos los registros. Las siguientes, solo inserta los vehículos nuevos, ignora los que ya existían y marca como baja los que han desaparecido, sin necesidad de vaciar y repoblar la tabla completa en cada ejecución.

Coordinador.py

El último script del proyecto es `coordinador.py`, y su papel es el de director de orquesta: no procesa ningún dato por sí mismo, sino que importa y ejecuta en el orden correcto los cuatro scripts anteriores, convirtiendo todo el pipeline en un proceso que puede lanzarse con un único comando.

La motivación para crear este script es sencilla. Los cuatro scripts que componen el backend, `descarga.py`, `procesarTXT.py`, `procesarCSV.py` e `insertarBBDD.py`, son un todo indivisible. No tiene sentido ejecutar uno sin los anteriores, y hacerlo manualmente cada mes introduciría el riesgo de olvidar algún paso, ejecutarlos en el orden equivocado o no darse cuenta de que uno de ellos ha fallado. El coordinador elimina todos esos riesgos centralizando el flujo en un único punto de entrada.

Cada script se ejecuta dentro de una función genérica que envuelve la llamada en un bloque de control de errores. Si el script completa su ejecución sin problemas, el coordinador informa de ello y pasa al siguiente. Si en cambio se produce cualquier error, el coordinador muestra el nombre del paso que ha fallado y el motivo, y detiene el proceso de inmediato con un código de salida que indica error.

Para la automatización mensual, el coordinador está configurado como una tarea diaria en PythonAnywhere, programada para ejecutarse a las tres de la madrugada. Sin embargo, lanzarlo cada día sería innecesario y podría causar problemas si los datos de la DGT aún no se han publicado. Para resolver esto, al inicio de cada ejecución el script comprueba si el día actual es uno de los días en los que debe actuar, concretamente el 5 y el 20 de cada mes. Si no lo es, el script termina sin hacer nada, informando simplemente de que ese día no corresponde ejecutar el proceso. Si sí lo es, arranca el pipeline completo. Elegimos esas dos fechas porque la DGT suele publicar los datos a mediados de mes, y tener dos intentos por mes nos da un margen de seguridad en caso de que la publicación se retrase o haya algún problema puntual en la primera fecha.

Esta lógica de control de días genera sin embargo una limitación práctica: si queremos ejecutar el proceso manualmente fuera de esas fechas, por ejemplo, para hacer pruebas o para una ejecución puntual, el script se detendría antes de hacer nada. Para evitar esto, implementamos el argumento opcional de línea de comandos. Cuando ejecutamos el coordinador con el comando: `python /home/DJF/scripts/coordinador.py --forzar`, el script detecta su presencia y omite la comprobación del día, ejecutando el pipeline completo independientemente de la fecha. La tarea programada de PythonAnywhere nunca incluye ese argumento, por lo que respeta siempre el control de días, mientras que nosotros lo usamos manualmente cuando lo necesitamos.

La configuración en PythonAnywhere es tan sencilla como añadir una tarea diaria en la pestaña de tareas programadas de la plataforma, indicando la ruta completa al script y la hora de

ejecución. A partir de ahí, el sistema funciona de forma completamente autónoma: cada día a las tres de la madrugada PythonAnywhere lanza el coordinador, que comprueba la fecha, actúa si corresponde y cierra el proceso, todo sin intervención humana.

3.4 Análisis diseño interfaz

Añadir a partir de la reunión del jueves 16. A parte de los prototipos, también tenemos que explicar los motivos de elegir JavaFX y los motivos de diseño. También explicaremos como conectamos JavaFX con la base de datos y la arquitectura que seguimos dentro de JavaFX.

3.4.1 Prototipo de Baja Fidelidad

3.4.2 Prototipo de Media Fidelidad

4. PLAN DE PROYECTO

4.1 Cronograma

4.2 Reparto tareas

- Arquitectura: David y Juan Francisco.
- PythonAnywhere: principalmente Juan Francisco. Intervención de David.
- Base de datos: principalmente David. Intervención de Juan Francisco.
- Interfaz JavaFX: David.
- Informe: Juan Francisco.

5. PRUEBAS

5.1 Índice casos de prueba

CASOS DE PRUEBAS	
TC-001	Inicio de sesión con credenciales correctas

5.2 Casos de prueba

6. DOCUMENTACIÓN DE LA APLICACIÓN

6.1 Manual de instalación

6.2 Demo

7. CONCLUSIONES

8. USO DE LA IA

Durante el desarrollo del proyecto utilizamos herramientas de inteligencia artificial como apoyo puntual para resolver dudas técnicas y explorar posibles soluciones, siempre desde un enfoque asistencial. Su papel se limitó a orientarnos y ofrecernos alternativas cuando surgían problemas complejos, sin intervenir en la creación del código ni en la toma de decisiones sobre la arquitectura o la lógica del sistema.

La IA nos resultó especialmente útil en el *backend*, donde la empleamos para aclarar cuestiones relacionadas con el entorno de despliegue en PythonAnywhere y, sobre todo, para trabajar en profundidad en la generación de la matrícula única mediante funciones de hash criptográfico. En este ámbito la utilizamos de forma muy conversacional: nos ayudó a entender mejor cómo funcionan los distintos algoritmos de hash, a valorar diferentes estrategias para generar las matrículas, a realizar cálculos aproximados sobre posibles duplicidades y a analizar el impacto de cada enfoque en la robustez del sistema. También recurrimos a ella para resolver problemas con *dataframes* en Python, principalmente en la corrección de errores y en la comprensión de su comportamiento interno. Además, nos sirvió para revisar y mejorar validaciones generales del código —como el uso de estructuras *try-catch*— y para pulir la claridad de los mensajes de salida y de los comentarios, buscando un código más legible y coherente.

9. BIBLIOGRAFÍA